

Task 1: Supervised Machine Learning Models

- **Regression** — lower MAE and higher R^2 is better. The Random Forest is expected to outperform Linear Regression if the relationship between features and traffic volume is non-linear (e.g. hour-of-day effects), at the cost of interpretability.
- **Classification** — no single metric tells the whole story for an imbalanced label like this one. Recall matters most if the goal is to catch as many high-risk hours as possible; precision matters if false alarms are costly; ROC AUC gives a threshold-independent view of separability.

Task 2: Unsupervised Machine Learning

Choice of k: Silhouette score peaks at $k=7$ (0.534), clearly above $k=4$ (0.460), so $k=7$ was used for the final clustering.

Cluster interpretation: Time of day is the dominant driver of traffic volume (overnight ≈ 900 < evening $\approx 2,500$ < midday $\approx 5,000$), with weather layered on top as a secondary effect — mild adverse weather barely reduces traffic in any time bucket, but the rarer, more severe/compound weather cluster (midday, severity 2.2) shows a real $\sim 15\%$ drop, distinct enough to form its own cluster.

Task 3: Deep Learning with Explainability

The LSTM predicts from a rolling window of recent history (including recent actual traffic_volume) while the Random Forest predicts from instantaneous calendar/weather features alone, so both target the same traffic_volume quantity. The LSTM's edge may partly reflect its access to recent traffic history rather than a purely superior model. The LSTM beat the Random Forest on both metrics — MAE 237.23 vs. 263.71, R^2 0.9665 vs. 0.9480. Time of day overwhelmingly dominates traffic volume, and weather is at best a secondary, and mostly negligible, factor

Task 4: Apply an Advanced AI Technique

Why they selected the technique: The task brief explicitly recommends MLflow because it connects directly to the MLOps component of the programme. Instead, it wraps the models already built in Task 1 (Linear/Logistic Regression, Random Forest Regressor/Classifier) with systematic experiment tracking: every run's parameters, metrics, and the trained model itself are logged automatically, rather than being copy-pasted by hand between notebook cells and a report.

How it was implemented: Each model from Task 1 is re-trained inside an ``mlflow.start_run()`` context. Model hyperparameters are logged with ``mlflow.log_param``, evaluation metrics with ``mlflow.log_metric``, and the fitted model itself as a versioned artifact with ``mlflow.sklearn.log_model``. Runs are grouped under a single named experiment (``metro_traffic_models``) so all four models regression and classification can be compared side by side afterwards, either through ``mlflow.search_runs()`` (used below to pull results back into this notebook) or the ``mlflow ui`` web dashboard.

What value it adds: it replaces manual, error-prone copy-pasting of metrics between runs with a queryable, versioned log every run's exact hyperparameters, metrics, and the resulting model artifact are all permanently recorded together, so results are fully reproducible and easy to compare later (e.g. "did Random Forest with max_depth=12 actually beat max_depth=8?"). It's also the natural first building block toward real MLOps: the logged model artifacts are already in a format (`mlflow.sklearn`) that can be registered and served without re-writing any code.

Limitations: MLflow only tracks and organizes experiments — it does not improve model performance, select features, or tune hyperparameters on its own. This notebook uses the default local file-based tracking store (`./mlruns/`), which is fine for a single-person project but doesn't support real team collaboration or remote access the way a hosted MLflow tracking server would. It also doesn't version the underlying dataset itself (a separate concern, e.g. DVC), so "reproducibility" here covers the model and its metrics, not guaranteeing the exact same `Features_Metro_Interstate_Traffic_Volume.csv` was used run to run unless that's tracked separately. Finally, logging discipline is manual nothing forces every future experiment to go through MLflow rather than being run ad hoc outside it.

Task 5: Build a Traffic Recommendation System

How it works: rather than just reporting historical averages, the system re-trains the Task 1 regressor (traffic volume) and classifier (accident-risk proxy) on the full engineered feature set, then generates a synthetic row for every hour of the day under the requested day type and weather, predicts both targets for each, and picks the lowest-traffic hour

with a safety check that flags (and, where possible, offers an alternative to) any recommended hour whose predicted accident-risk is still notably elevated.

Task 6: MLOps and Deployment Simulation

The MLOps and Deployment Simulation was carried out on the Metro Interstate Traffic Volume dataset, covering the full workflow from experimentation to deployment. Model versioning was performed by training and comparing three candidate versions of the traffic-volume regressor, and experiment tracking was implemented using MLflow to log parameters, metrics, and model versions across experiments. A FastAPI deployment mock-up was then built to serve the production model, exercised via FastAPI's TestClient rather than a running server process. The deployed model was further monitored across three sample data sets to check for feature-distribution and prediction-error drift, and the resulting PASS/ALERT status for each check was tabulated into a final overall system status dashboard.